Student name:
CSC 3449-07.
Instructor: Rodrigo Canaan

Shortest Path in Weighted Graphs

Introduction

We previously used **Breadth-First Search (BFS)** to find the *distance* between a selected root vertex and every reachable vertex in unweighted graphs. We defined distance as the number of edges in the shortest path between two vertices. We need to update our definitions if we want to work with graphs with weighted edges.

Definition 1 - Path Cost: The cost of a path from u to v is the sum of the edge weights along that path.

Definition 2 - Distance: The distance between two vertices u and v is the minimum cost among all paths from u to v, assuming v is reachable from u.

$$\delta(u, v) = min_{P} \{cost(P) \mid P \ \ is\ a\ path\ from\ u\ to\ v)\}$$

By convention, we define distance to be infinite if v is not reachable.

This definition works for both directed and undirected weighted graphs. The length of the shortest path found by BFS can be seen as a special case of this new notion of distance if all edges have a weight of 1.

Exercise 1: Draw a graph with weighted edges, and choose a root vertex r and a target vertex t such that the path with the *fewest number* of edges from r to t is not the same as the path with the *minimum cost* as defined above.

Student name:
CSC 3449-07.
Instructor: Rodrigo Canaan

For now, assume G is a weighted graph with no negative edge weights, and we will refer to a path's cost as its *length* and the minimum-cost path as the *shortest path* for simplicity.

Relaxing an edge

We will call a vertex whose true distance from the root (and the corresponding shortest path) is known *settled*. A key observation is that if u is settled and there's an edge from u to v, that gives us a candidate shortest path from r to v, and the length of that path is an upper bound for the distance from r to v:

$$\delta(r,u) + w(u,v) \leq \delta(r,v)$$

We can use this observation to maintain and update an estimate (upper bound) of the distance from the root to each vertex. We call this process *relaxing* the edge (u,v):

Relax (u,v)

Input: A settled vertex u and a neighbor of u, v
Output: An update to the estimate of $\delta(r,v)$
1. if $dist[u] + w(u,v) < dist[v]$
    1.1. $dist[v] \leftarrow dist[u] + w(u,v)$
    1.2. $parent[v] \leftarrow u$

In the code, dist[v] contains our current estimate of $\delta(r,v)$ for a predetermined root. and it can decrease (but never increase) after relaxing an incoming edge to v.

The parent pointer indicates that, in the shortest path found so far from r to u, the last edge is from *parent[v]* to *v* . Once v is settled, we can reconstruct the shortest path from r to v by recursively following this pointer.

Student name:
CSC 3449-07.
Instructor: Rodrigo Canaan

Settling a vertex

We will use the following greedy heuristic to choose which vertex to settle: **Settle the unsettled vertex with minimum distance estimate**. Once we settle a vertex, we relax all of its edges. Our algorithm, therefore, looks like this:

WeightedShortestPath (G, r)

Input: a graph with non-negative weighted edges $G = \{V, E, w\}$ and a chosen root r
Output: the array $dist$ holding the distance of each vertex to r, and the array *parent* containing the parent pointers

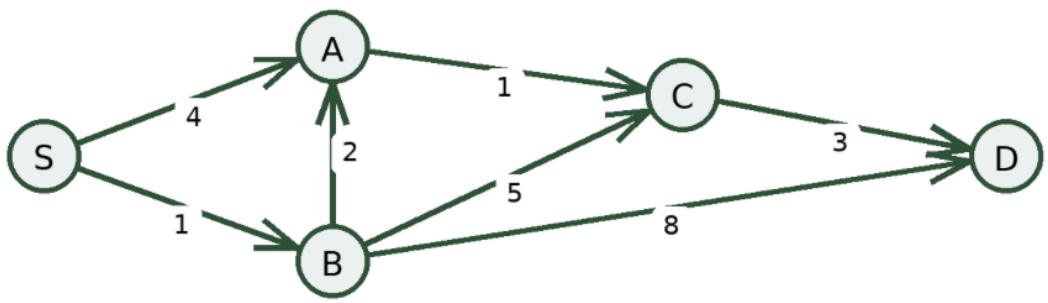
```
// Initialization of all vertices
1. for all v ∈ V:
        1.1.dist[v] ← ∞
        1.2.settled[v] ← False
        1.3.parent[v] ← None

// Initializing the root
2. dist[r] ← 0

// main loop

3. For i in 1 to n:
        3.1.let u be an unsettled vertex with minimum distance
        3.2.if dist[u] = ∞: break
        3.3.settled[u] ← True
        3.4.For v in u.out:
                3.4.1.Relax(u,v)
4. return dist, parent
```

Exercise 2: Consider the graph below:



Student name:
CSC 3449-07.
Instructor: Rodrigo Canaan

Starting with source S, perform the sequence of relaxations that the algorithm would make

Step	Selected vertex	Edges relaxed	Distances updated	Parent changes
0				
1				
2				
3				
4				
5				